

Chapter 3.2

FOR Loop

1. What Is a FOR Loop?

A for loop is used when you know in advance how many times you want to repeat a block of code — for example, running a block exactly 5 times, or from 1 to 10.

Syntax

```
for (initialization; condition; update) {  
    // statements  
}
```

The loop has three parts, all controlled in a single line:

Part	Purpose
initialization	Runs once, before the loop starts (e.g., <code>int i = 1</code>) — sets the starting value of the loop counter.
condition	Checked before every iteration; the loop keeps running as long as this is true.
update	Runs after each iteration (e.g., <code>i++</code> or <code>i--</code>) — changes the counter until the condition becomes false.

2. Basic FOR Loop Example — Counting Up

Logic

- Start a counter `i` at 1.
- Keep looping while `i <= 5`.
- Print the counter value, then increase `i` by 1 (`i++`) after each pass.

Java Code

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Count: " + i);  
        }  
    }  
}
```

Output: Count: 1, Count: 2, Count: 3, Count: 4, Count: 5

3. FOR Loop Example — Sum of Numbers (1 to 10)

Logic

- Declare a variable `sum` and initialize it to 0 before the loop.
- Loop `i` from 1 to 10.

- On each pass, add *i* to sum using the shorthand `sum += i` (same as `sum = sum + i`).
- After the loop ends, print the final total.

Java Code

```
public class ForLoopExample3 {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 10; i++) {
            sum += i;
        }
        System.out.println("Sum = " + sum);
    }
}
```

Output: Sum = 55

4. Countdown (Decrement Loop)

Logic

- Start the counter *i* at 5 instead of 1.
- Keep looping while *i* > 0.
- Instead of increasing, decrease *i* by 1 each time (`i--`), so the loop counts downward.
- After the loop finishes, print a final message.

Java Code

```
public class Countdown {
    public static void main(String[] args) {
        for (int i = 5; i > 0; i--) {
            System.out.println("Countdown: " + i);
        }
        System.out.println("Liftoff!");
    }
}
```

Output: Countdown: 5, Countdown: 4, Countdown: 3, Countdown: 2, Countdown: 1, Liftoff!

5. FOR Loop with a Condition — Printing Odd Numbers

A for loop can be combined with an if statement inside its body to filter which values get processed — here, only the odd numbers between 1 and 10 are printed.

Logic

- Loop *i* from 1 to 10.
- Inside the loop, use the modulus operator (%) to check the remainder when *i* is divided by 2.
- If `i % 2 != 0`, the number is odd, so print it. Even numbers are skipped since the if condition is false for them.

Java Code

```
public class OddNumbers {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i % 2 != 0) {  
                System.out.println(i + " is odd");  
            }  
        }  
    }  
}
```

Output: 1 is odd, 3 is odd, 5 is odd, 7 is odd, 9 is odd

6. Key Takeaways

- Use a for loop when the number of repetitions is known or countable in advance.
- The counter can increment (i++) or decrement (i--), and can start/end at any values you choose.